

Exercício – Adicionar um controlador

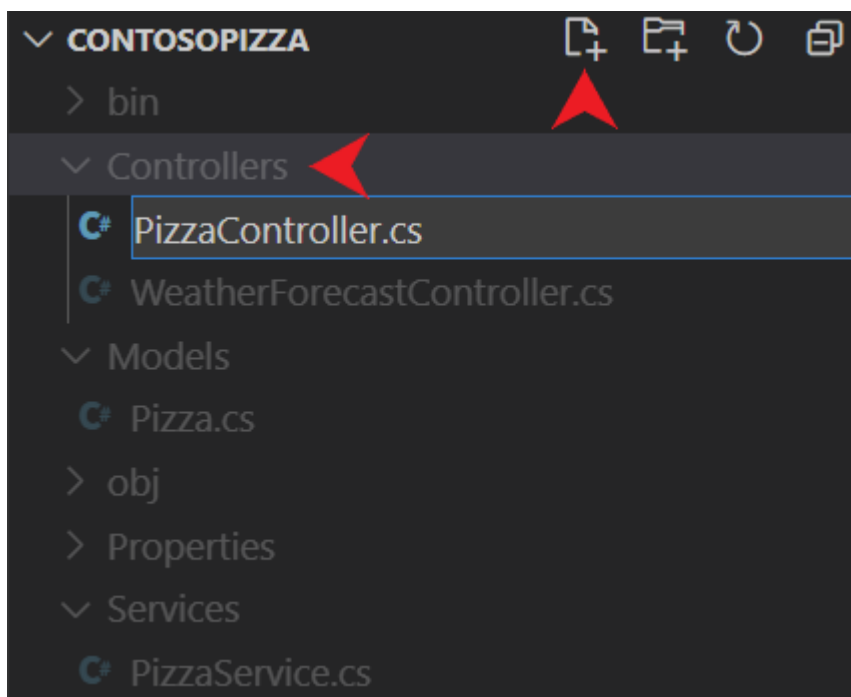
Concluído

- 5 minutos

Um *controlador* é uma classe pública com um ou mais métodos públicos conhecidos como *ações*. Por convenção, um controlador é colocado no diretório *Controllers* da raiz do projeto. As ações são expostas como endpoints HTTP dentro do controlador da Web API.

Criar um controlador

1. Selecione a pasta *Controladores* no Visual Studio Code e adicione um novo arquivo chamado *PizzaController.cs*.



Um arquivo de classe vazio chamado *PizzaController.cs* é criado no diretório *Controladores*. O nome do diretório *Controladores* é uma convenção. O nome do diretório vem da arquitetura model-view-*controller* que a API Web usa.

Observação

Por convenção, os nomes de classe do controlador apresentam *Controlador* como sufixo.

2. Adicione o código a seguir a *Controllers/PizzaController.cs*. Salve suas alterações.

C#

```
using ContosoPizza.Models;
using ContosoPizza.Services;
using Microsoft.AspNetCore.Mvc;

namespace ContosoPizza.Controllers;

[ApiController]
[Route("[controller]")]
public class PizzaController : ControllerBase
{
    public PizzaController()
    {
    }

    // GET all action

    // GET by Id action
```

Conforme você aprendeu anteriormente, essa classe deriva de `ControllerBase`, a classe base para trabalhar com solicitações HTTP no ASP.NET Core. Ele também inclui os dois atributos padrão sobre os quais você aprendeu: `[ApiController]` e `[Route]`. Assim como antes, o atributo `[Route]` define um mapeamento para o token `[controller]`. Como essa classe de controlador é nomeada `PizzaController`, esse controlador manipula as solicitações para `https://localhost:{PORT}/pizza`.

Obter todas as pizzas

O primeiro verbo REST que precisamos implementar é `GET`, em que um cliente pode obter todas as pizzas da API. Você pode usar o atributo `[HttpGet]` interno para definir um método que retorne as pizzas de nosso serviço.

Substitua o comentário `// GET all action` em `Controllers/PizzaController.cs` pelo seguinte código:

C#

```
[HttpGet]
public ActionResult<List<Pizza>> GetAll() =>
```

```
PizzaService.GetAll();
```

A ação anterior:

- Responde apenas ao verbo HTTP `GET` , conforme indicado pelo atributo `[HttpGet]` .
- Retorna uma instância de `ActionResult` do tipo `List<Pizza>` . O tipo `ActionResult` é a classe base para todos os resultados da ação no ASP.NET Core.
- Consulta o serviço para todas as pizzas e retorna automaticamente os dados com um valor `Content-Type` igual a `application/json` .

Recuperar apenas uma pizza

O cliente também pode querer solicitar informações sobre uma pizza específica em vez de toda a lista. Você pode implementar outra ação `GET` que requer um parâmetro `id` . Você pode usar o atributo `[HttpGet("{id}")]` interno para definir um método que retorne as pizzas de nosso serviço. A lógica de roteamento registra `[HttpGet]` (sem `id`) e `[HttpGet("{id}")]` (com `id`) como duas rotas diferentes. Você pode então criar uma ação separada para recuperar um único item. Substitua o comentário `// GET by Id action` em `Controllers/PizzaController.cs` pelo seguinte código:

C#

```
[HttpGet("{id}")]
public ActionResult<Pizza> Get(int id)
{
    var pizza = PizzaService.Get(id);

    if(pizza == null)
        return NotFound();

    return pizza;
}
```

A ação anterior:

- Responde apenas ao verbo HTTP `GET` , conforme indicado pelo atributo `[HttpGet]` .
- Requer que o valor do parâmetro `id` seja incluído no segmento da URL após `pizza/` . Lembre-se, o atributo `[Route]` do nível do controlador definiu o padrão `/pizza` .
- Consulta o banco de dados para uma pizza que corresponde ao parâmetro `id` fornecido.

Cada instância de `ActionResult` usada na ação anterior é mapeada para o código de status HTTP correspondente na tabela a seguir:

ASP.NET Core resultado da ação	Código de status HTTP	Descrição
<code>Ok</code> está implícito	200	Um produto que corresponde ao parâmetro <code>id</code> fornecido existe no cache na memória. O produto está incluído no corpo da resposta no tipo de mídia,

		conforme definido no cabeçalho da solicitação HTTP <code>accept</code> (JSON por padrão).
NotFound	404	Um produto que corresponde ao parâmetro <code>id</code> fornecido não existe no cache na memória.

Compilar e executar o novo controlador

Compile e inicie a API Web executando o seguinte comando:

CLI do .NET

```
dotnet run
```

Testar o controlador com um arquivo HTTP

1. Abrir **ContosoPizza.http**
2. Adicione um novo **GET** para chamar o endpoint `Pizza` abaixo do separador **###**:

Saída

```
GET {{ContosoPizza_HostAddress}}/pizza/  
Accept: application/json  
  
###
```

3. Selecione o comando **Enviar Solicitação** acima dessa nova chamada **GET**.

O comando anterior retorna uma lista de todas as pizzas em JSON:

Saída

```
HTTP/1.1 200 OK  
Connection: close  
Content-Type: application/json; charset=utf-8  
Date: Wed, 17 Jan 2024 16:57:09 GMT  
Server: Kestrel  
Transfer-Encoding: chunked  
  
[  
  {  
    "id": 1,  
    "name": "Classic Italian",  
    "isGlutenFree": false  
  },  
  {  
    "id": 2,  
    "name": "Veggie",  
    "isGlutenFree": true  
  }  
]
```

4. Para consultar apenas uma pizza, você pode fazer outra solicitação `GET`, mas passar um parâmetro `id` usando o seguinte comando:

Saída

```
GET {{ContosoPizza_HostAddress}}/pizza/1  
Accept: application/json  
  
###
```

O comando anterior retorna `Classic Italian` com a seguinte saída:

Saída

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Fri, 02 Apr 2021 21:57:57 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "id": 1,
  "name": "Classic Italian",
  "isGlutenFree": false
}
```

5. Nossa API também lida com situações em que o item não existe. Chame a API novamente, mas passe um parâmetro pizza `id` inválido usando o seguinte comando:

Saída

```
GET {{ContosoPizza_HostAddress}}/pizza/5
Accept: application/json

###
```

O comando anterior retorna um erro `404 Not Found` com a seguinte saída:

Saída

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json; charset=utf-8
Date: Fri, 02 Apr 2021 22:03:06 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "type":
    "https://tools.ietf.org/html/rfc7231#section-6.5.4",

```

```
    "title": "Not Found",  
    "status": 404,  
    "traceId": "00-ec263e401ec554b6a2f3e216a1d1fac5-  
4b40b8023d56762c-00"  
}
```

Agora que você terminou de implementar os verbos `GET`. Na próxima unidade, você pode adicionar mais ações a `PizzaController` para dar suporte a operações CRUD em dados de pizza.

Opcional: Testar o controlador com o REPL (loop de leitura-avaliação-impressão HTTP) HTTP da Linha de Comando

1. Abra o terminal `httprepl` existente ou um novo terminal integrado do Visual Studio Code selecionando **Terminal>Novo Terminal** no menu principal.
2. Conecte-se à API Web executando o seguinte comando:

CLI do .NET

```
httprepl https://localhost:{PORT}
```

Outra opção é executar o seguinte comando a qualquer momento enquanto `HttpRepl` estiver em execução:

CLI do .NET

```
connect https://localhost:{PORT}
```

3. Para ver o ponto de extremidade de `Pizza` recém-disponível, execute o seguinte comando:

CLI do .NET

```
ls
```

O comando anterior detecta todas as APIs disponíveis no ponto de extremidade conectado. Ele deve exibir o seguinte código:

Saída


```
https://localhost:{PORT}/> ls
.
Pizza
WeatherForecast
```

4. Acesse o endpoint `Pizza` executando o seguinte comando:

CLI do .NET

```
cd Pizza
```

O comando anterior mostra uma saída das APIs disponíveis para o ponto de extremidade `Pizza`:

Saída

```
https://localhost:{PORT}/> cd Pizza
/Pizza [GET]
```

5. Faça uma solicitação `GET` no `HttpRepl` usando o seguinte comando:

CLI do .NET

```
get
```

O comando anterior retorna uma lista de todas as pizzas em JSON:

Saída

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Fri, 02 Apr 2021 21:55:53 GMT
Server: Kestrel
Transfer-Encoding: chunked

[
  {
    "id": 1,
    "name": "Classic Italian",
```

```
        "isGlutenFree": false
      },
      {
        "id": 2,
        "name": "Veggie",
        "isGlutenFree": true
      }
    ]
  }
}
```

6. Para consultar apenas uma pizza, você pode fazer outra solicitação `GET`, mas passar um parâmetro `id` usando o seguinte comando:

CLI do .NET

```
get 1
```

O comando anterior retorna `Classic Italian` com a seguinte saída:

Saída

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Fri, 02 Apr 2021 21:57:57 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "id": 1,
  "name": "Classic Italian",
  "isGlutenFree": false
}
```

7. Nossa API também lida com situações em que o item não existe. Chame a API novamente, mas passe um parâmetro pizza `id` inválido usando o seguinte comando:

CLI do .NET

```
get 5
```

O comando anterior retorna um erro `404 Not Found` com a seguinte saída:

Saída

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json; charset=utf-8
Date: Fri, 02 Apr 2021 22:03:06 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "type":
  "https://tools.ietf.org/html/rfc7231#section-6.5.4",
  "title": "Not Found",
  "status": 404,
  "traceId": "00-ec263e401ec554b6a2f3e216a1d1fac5-
4b40b8023d56762c-00"
}
```

8. Retorne ao terminal `dotnet` na lista suspensa do Visual Studio Code e encerre a API da Web pressionando Ctrl+C no teclado.

Agora que você terminou de implementar os verbos `GET`. Na próxima unidade, você pode adicionar mais ações a `PizzaController` para dar suporte a operações CRUD em dados de pizza.